

```

/* =====
WealthFlow Autonomous Module v1.0
-----
Client-side conductor for the robotic financial OS:

    • Card Registry (card_last4 → bank + type + name) stored in DB.settings
    • Pending-queue (IndexedDB) for offline self-healing
    • Manual + auto Predictive Wealth refresh
    • Apply-routed-transaction helper used by SMS ingestion + manual paste
    • FIFO reconciler dispatcher for "credit to CC" events
    • OCR receipt → SMS-match auto-linker

All functions are attached to window so onclick handlers can reach them.
=====*/

(function () {
    'use strict';

    if (window.WF_AUTONOMOUS_LOADED) return;
    window.WF_AUTONOMOUS_LOADED = '1.0';

    // -----
    // 1. Card Registry
    // -----
    function getCardRegistry() {
        try {
            const s = (window.DB && DB.getObj && DB.getObj('settings')) || {};
            return s.cardRegistry || {};
        } catch (_) { return {}; }
    }
    function setCardRegistry(reg) {
        const s = (window.DB && DB.getObj && DB.getObj('settings')) || {};
        s.cardRegistry = reg;
        DB.setObj('settings', s);
        if (typeof window.syncToCloud === 'function') {
            try { syncToCloud(); } catch (_) {}
        }
        return reg;
    }
    function upsertCard(last4, fields) {
        last4 = String(last4 || '').trim();
        if (!/^\d{4}$/.test(last4)) {
            if (typeof window.notify === 'function') notify('Card last-4 must be
            return null;

```

```

    }
    const reg = getCardRegistry();
    reg[last4] = Object.assign({}, reg[last4] || {}, fields, {
        last4, updated_at: Date.now()
    });
    setCardRegistry(reg);
    if (typeof window.notify === 'function') notify(`✓ Card •••${last4} saved`);
    return reg[last4];
}

function deleteCard(last4) {
    const reg = getCardRegistry();
    if (reg[last4]) { delete reg[last4]; setCardRegistry(reg); }
    return reg;
}

window.wfCardRegistry = { get: getCardRegistry, set: setCardRegistry, upsert:

// -----
// 2. Offline queue (IndexedDB) – processed-hash dedup store
// [v7.8.0] Device token removed (was SMS-forwarder auth only).
// -----

const IDB_NAME = 'wfAutonomous';
const IDB_VER = 1;
function openIDB() {
    return new Promise((resolve, reject) => {
        const req = indexedDB.open(IDB_NAME, IDB_VER);
        req.onupgradeneeded = () => {
            const db = req.result;
            if (!db.objectStoreNames.contains('queue')) {
                db.createObjectStore('queue', { keyPath: 'id', autoIncrement:
            }
            if (!db.objectStoreNames.contains('processed_hashes')) {
                db.createObjectStore('processed_hashes', { keyPath: 'hash' })
            }
        };
        req.onsuccess = () => resolve(req.result);
        req.onerror = () => reject(req.error);
    });
}

// [v7.8.0] enqueue retained as a no-op; nothing enqueues now that SMS is gone
async function enqueue() { return false; }
// [v7.8.0] SMS removed – the offline queue previously held SMS-ingest
// payloads. With SMS gone the queue is unused; drain is a harmless no-op
// kept only so window.wfQueue.drain() callers still resolve cleanly.
async function drainQueue() {
    return { drained: 0, remaining: 0 };
}

async function alreadyProcessed(hash) {

```

```

    try {
      const db = await openIDB();
      return new Promise(res => {
        const tx = db.transaction('processed_hashes', 'readonly');
        tx.objectStore('processed_hashes').get(hash).onsuccess = (ev) =>
          res(!ev.target.result);
      });
    } catch (_) { return false; }
  }
}

async function markProcessed(hash) {
  try {
    const db = await openIDB();
    await new Promise(res => {
      const tx = db.transaction('processed_hashes', 'readwrite');
      tx.objectStore('processed_hashes').put({ hash, ts: Date.now() });
      tx.oncomplete = res;
    });
  } catch (_) {}
}

window.wfQueue = { enqueue, drain: drainQueue };

// -----
// 4. Apply Brain Result – writes the classified transaction to the
// correct WealthFlow module via DB.set.
// -----

async function applyBrainResult(brain, opts) {
  opts = opts || {};
  if (!brain || !brain.ok || !brain.classified) return { ok: false, reason:
  if (!opts.skipDedup && await alreadyProcessed(brain.hash)) {
    return { ok: false, reason: 'duplicate (already applied)' };
  }

  const routed = brain.routed || {};
  const fields = routed.suggested_fields || {};
  let module = opts.forceModule || routed.module;

  // — v7.7.0 Intelligence layer -----
  // Runs BEFORE generic routing, but is skipped when re-applying from the
  // Quarantine Zone (the user has already made the decision there).
  if (!opts.skipIntel) {
    // (B) Semantic Goal / Loan allocation – highest priority. If the
    // transaction description matches a savings goal or a loan by
    // name, route the money straight there and stop.
    if (typeof window.wfTrySemanticAllocate === 'function') {
      try {
        const alloc = await window.wfTrySemanticAllocate(brain);
        if (alloc && alloc.ok) {

```

```

        await markProcessed(brain.hash);
        if (typeof window.syncToCloud === 'function') { try { syn
        return alloc;
        }
    } catch (e) { console.warn('[Autonomous] semantic hook error:', e
}

// (C) Confidence gate → Quarantine Zone. If the brain isn't at
//     least 95% sure, DO NOT guess – ask the user.
const rConf = (routed.confidence !== null) ? routed.confidence : 1;
const mConf = (brain.resolved_merchant && brain.resolved_merchant.con
    ? brain.resolved_merchant.confidence : 1;
const conf = Math.min(rConf, mConf);
if (conf < 0.95 && typeof window.wfQuarantineAdd === 'function') {
    window.wfQuarantineAdd(brain, 'AI only ' + Math.round(conf * 100)
    await markProcessed(brain.hash); // hold it; the quarantine entry
    if (typeof window.notify === 'function') {
        notify('🚨 1 transaction needs your review (low confidence)',
    }
    return { ok: true, module: 'quarantine' };
}
}

try {
    if (module === 'goal') {
        // forced from Quarantine "Goal" chip – needs a target id in fiel
        const arr = (DB.get('targets') || []);
        const gi = arr.findIndex(x => x.id === fields.goalId);
        if (gi < 0) return { ok: false, reason: 'goal not found' };
        if (!Array.isArray(arr[gi].savings)) arr[gi].savings = [];
        arr[gi].savings.push({ id: 'auto_' + Date.now().toString(36), amo
        DB.set('targets', arr);
    } else if (module === 'loan') {
        const arr = (DB.get('loans') || []);
        const li = arr.findIndex(x => x.id === fields.loanId);
        if (li < 0) return { ok: false, reason: 'loan not found' };
        if (!Array.isArray(arr[li].payments)) arr[li].payments = [];
        arr[li].payments.push({ id: 'auto_' + Date.now().toString(36), am
        DB.set('loans', arr);
    } else if (module === 'income') {
        const arr = (DB.get('income') || []);
        arr.push({
            id: 'auto_' + Date.now().toString(36),
            source: fields.source || 'Auto',
            amount: fields.amount, date: fields.date,
            notes: fields.notes || '', auto: true,
            hash: brain.hash, createdAt: new Date().toISOString()

```

```

    });
    DB.set('income', arr);
  } else if (module === 'expenses') {
    const arr = (DB.get('expenses') || []);
    arr.push({
      id: 'auto_' + Date.now().toString(36),
      desc: fields.desc, amount: fields.amount,
      cat: fields.cat || 'Other',
      date: new Date(fields.date).toISOString().slice(0,10),
      date_ms: fields.date, completed: false,
      notes: fields.notes || '', auto: true,
      hash: brain.hash, createdAt: new Date().toISOString()
    });
    DB.set('expenses', arr);
  } else if (module === 'cconetime') {
    const arr = (DB.get('cconetime') || []);
    arr.push({
      id: 'auto_' + Date.now().toString(36),
      desc: fields.desc, amount: fields.amount,
      date: new Date(fields.date).toISOString().slice(0,10),
      date_ms: fields.date,
      bank: fields.bank || '', card_last4: fields.card_last4,
      type: fields.type || 'purchase',
      notes: fields.notes || '', completed: false, auto: true,
      hash: brain.hash, createdAt: new Date().toISOString()
    });
    DB.set('cconetime', arr);
  } else if (module === 'subscriptions') {
    const arr = (DB.get('subscriptions') || []);
    // Skip if a sub with same name already exists
    if (!arr.some(s => (s.name||'').toLowerCase() === (fields.name||'
      arr.push({
        id: 'auto_' + Date.now().toString(36),
        name: fields.name, category: fields.category,
        amount: fields.amount, dueDay: fields.due_day,
        cycle: fields.cycle || 'monthly', auto: true,
        hash: brain.hash, createdAt: new Date().toISOString()
      });
      DB.set('subscriptions', arr);
    }
  } else if (module === 'cc_payment') {
    // Trigger FIFO reconciliation
    return await runFifoReconcile(fields);
  } else {
    return { ok: false, reason: 'Unknown module: ' + module };
  }
}

```

```

    await markProcessed(brain.hash);
    if (typeof window.syncToCloud === 'function') {
        try { syncToCloud(); } catch (_) {}
    }
    if (typeof window.notify === 'function') {
        notify(`🤖 Auto-logged: ${brain.resolved_merchant} && brain.resolv
    }
    return { ok: true, module };
} catch (e) {
    console.error('[Autonomous] applyBrainResult error:', e);
    return { ok: false, reason: e.message };
}
}
window.wfApplyBrainResult = applyBrainResult;

// -----
// 5. FIFO Reconciler - picks up "credit to CC" payment events
// -----
async function runFifoReconcile({ amount, card_last4, timestamp }) {
    // Gather all unsettled debits for this card
    const ccot = (DB.get('cconetime') || [])
        .filter(x => x && x.card_last4 === card_last4 && !x.completed)
        .map(x => ({
            id: x.id, module: 'cconetime',
            card_last4: x.card_last4, amount: x.amount,
            timestamp: x.date_ms || new Date(x.date).getTime()
        }));
    const cci = (DB.get('ccinstall') || [])
        .filter(x => x && x.card_last4 === card_last4 && !x.completed)
        .map(x => ({
            id: x.id, module: 'ccinstall',
            card_last4: x.card_last4, amount: x.amount,
            timestamp: x.date_ms || new Date(x.date).getTime()
        }));
    const debits = ccot.concat(cci);

    if (!debits.length) {
        if (typeof window.notify === 'function') notify(`🇺🇸 CC payment receiv
        return { ok: true, settled: 0 };
    }

    try {
        const r = await fetch('/api/fifo-reconcile', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                card_last4, payment_amount: amount, payment_ts: timestamp, de

```

```

    })
  });
  const plan = await r.json();
  if (!plan.ok) throw new Error(plan.error || 'reconcile failed');

  // Apply settled flags
  const settledIds = new Set((plan.settled || []).map(s => s.id));
  ['cconetime', 'ccinstall'].forEach(mod => {
    const arr = DB.get(mod) || [];
    let mutated = false;
    for (const item of arr) {
      if (settledIds.has(item.id)) {
        item.completed = true;
        item.settled_at = plan.payment_ts;
        item.settled_by_payment = true;
        mutated = true;
      }
      if (plan.partial && plan.partial.id === item.id) {
        item.amount = plan.partial.remaining;
        item.partial_paid = (item.partial_paid || 0) + plan.parti
        mutated = true;
      }
    }
    if (mutated) DB.set(mod, arr);
  });

  if (typeof window.syncToCloud === 'function') {
    try { syncToCloud(); } catch (_) {}
  }
  if (typeof window.notify === 'function') {
    const n = plan.settled.length;
    notify(`🇬🇧 FIFO reconciled: ${n} CC charge${n !== 1 ? 's': ''} cleared`
  )
  }
  return { ok: true, settled: plan.settled.length, partial: plan.partia
} catch (e) {
  console.error('[FIFO] error:', e);
  if (typeof window.notify === 'function') notify('FIFO reconcile faile
  return { ok: false, error: e.message };
}
}

window.wfFifoReconcile = runFifoReconcile;

// _____
// 7. Predictive Wealth refresh
// _____

async function refreshPredictions(horizon) {
  try {

```

```

    const r = await fetch('/api/predict-wealth', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        expenses: DB.get('expenses') || [],
        income: DB.get('income') || [],
        subscriptions: DB.get('subscriptions') || [],
        horizon_months: horizon || 12
      })
    });
    const data = await r.json();
    if (data.ok) {
      const s = DB.getObj('settings') || {};
      s.lastPredictions = { ...data, refreshed_at: Date.now() };
      DB.setObj('settings', s);
    }
    return data;
  } catch (e) { return { ok: false, error: e.message }; }
}
window.wfRefreshPredictions = refreshPredictions;

// _____
// 8. FX helper
// _____
async function fxConvert(amount, from, to) {
  try {
    const r = await fetch('/api/fx-rate', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ amount, from, to })
    });
    return await r.json();
  } catch (e) { return { ok: false, error: e.message }; }
}
window.wfFxConvert = fxConvert;

// _____
// 9. Boot – refresh predictions once on load (silent).
// [v7.8.0] SMS removed: self-healing queue, share-target inbox, and
// server-inbox pollers all deleted. They existed only to deliver SMS
// payloads. The autonomous engine (applyBrainResult, FIFO, dedup, card
// registry, predictions) remains – driven by PDF/photo statement import
// and the Intelligence layer (Vault, semantic allocation, quarantine).
// _____
if (typeof document !== 'undefined') {
  document.addEventListener('DOMContentLoaded', () => {
    setTimeout(() => { refreshPredictions(12).catch(() => {}); }, 8000);
  });
}

```



```
    });  
  }  
  
  console.log('[Autonomous]  WealthFlow Autonomous Module v1.1 loaded (SMS-fr  
}) ();
```